

CTRL-REPORT-002: Сервис работы с табличными отчетами

ID: CTRL-REPORT-002
Название: [Feature] Сервис CRUD для работы с табличными отчетами (Excel) с поддержкой обновлений и статистики качества
Тип: Feature
Приоритет: High
Статус: Backlog
Назначен: - Эпик: Reports Management
Спринт: -
Story Points: 21

Саммари

Разрабатывается сервис для работы с табличными отчетами (CRUD операции) с поддержкой загрузки Excel файлов с произвольным набором столбцов, обновления данных (полная замена или дозапись), а также расчета статистики качества данных. Сервис должен работать с обоими фреймворками - FastAPI (REST API) и FastStream (асинхронная обработка) через универсальные хендлеры.

Технологический стек:

- **Backend:** Python 3.11+, FastAPI, FastStream
- **База данных:** PostgreSQL с async SQLAlchemy ORM
- **Архитектура:** Clean Architecture, SOLID принципы
- **Тестирование:** pytest, pytest-asyncio (без моков, только реальные объекты)
- **Миграции:** Alembic
- **Валидация:** Pydantic v2
- **Обработка Excel:** pandas, openpyxl

Ключевые особенности:

- Хранение табличных данных в нормализованном виде (отдельная таблица для строк)
- Поддержка произвольного набора столбцов (динамическая схема)
- Два режима обновления: полная замена таблицы или дозапись новых данных
- Расчет статистики качества: новые строки, пустые значения, уникальные значения
- Внешние ключи на user_id и template_id (без FK constraints в БД)

ПОЛЬЗОВАТЕЛЬСКАЯ ИСТОРИЯ

Как **пользователь системы**
Я хочу **загружать Excel файлы с табличными данными, обновлять их и получать статистику качества**
Чтобы **эффективно управлять табличными отчетами и отслеживать изменения данных**

ОПИСАНИЕ

Создание полнофункционального сервиса для работы с табличными отчетами, включающего:

- CRUD операции (Create, Read, Update, Delete)
- Загрузку Excel файлов с произвольным набором столбцов
- Обновление данных с поддержкой полной замены или дозаписи
- Получение данных в формате Excel или JSON
- Расчет статистики качества данных (новые строки, пустые значения, уникальные значения)
- Универсальные хендлеры для работы с FastAPI и FastStream

ТРЕБОВАНИЯ ИЗ CONVENTIONS.MD

Стиль кода

- PEP 8 - основной стандарт стиля Python
- Black - автоформатирование с длиной строки 120 символов
- isort - сортировка импортов по профилю "wemake"
- Type hints - обязательная типизация всех функций и методов

Именованние

- Классы - PascalCase (TableReportService, TableReportRepository)
- Функции и переменные - snake_case (create_table_report, report_id)
- Константы - UPPER_SNAKE_CASE (MAX_ROWS_PER_BATCH)
- Приватные методы - префикс _ (_validate_excel_file)
- Файлы и модули - snake_case (table_report_service.py)

Документирование кода

Все функции и методы должны иметь docstrings с описанием Args, Returns, Raises.

Обработка ошибок

- Явная обработка исключений

- Логирование ошибок с контекстом
- Запрещено скрытие исключений (try/except: pass)

Асинхронный код

- Использовать `async/await` для всех I/O операций
- Группировка `async` операций через `asyncio.gather`
- Запрещены синхронные I/O операции в `async` функциях

Логирование

- Структурированное логирование с контекстом (extra параметры)
- Логирование ошибок с `exc_info=True`
- Запрещено логирование паролей/токенов

Архитектурные слои

```
app/
├─ api/           # Слой представления (Controllers)
├─ services/      # Слой бизнес-логики (Use Cases)
├─ clients/       # Слой доступа к данным (Repositories & External APIs)
├─ config/        # Конфигурация
└─ templates/     # Ресурсы
```

Правила зависимостей

1. API слой может зависеть от Services
2. Services слой может зависеть от Clients
3. Clients слой не должен зависеть от верхних слоев
4. Config может использоваться всеми слоями

Запрещенные практики

1. Прямые SQL запросы - использовать только SQLAlchemy
2. Синхронные I/O операции - только `async/await`
3. Глобальные переменные - использовать `dependency injection`
4. Хардкод конфигурации - только через `Settings`
5. Игнорирование исключений - всегда обрабатывать ошибки
6. Прямое обращение к БД из API слоя - только через сервисы
7. Изменение схемы БД без миграций - только через `Alembic`

ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ

1. CRUD операции

- **Create:** Создание табличного отчета из Excel файла
- **Read:** Получение табличного отчета в формате Excel или JSON
- **Update:** Обновление данных с поддержкой полной замены или дозаписи
- **Delete:** Удаление табличного отчета

2. Структура данных табличного отчета

Метаданные отчета (таблица `table_reports`)

- `id` (int, PK, autoincrement) - уникальный идентификатор
- `name` (str, required) - наименование отчета
- `created_at` (datetime) - дата создания
- `updated_at` (datetime) - дата последнего обновления
- `user_id` (str, required) - внешний ключ на пользователя (без FK constraint)
- `template_id` (int, optional) - внешний ключ на шаблон (без FK constraint)
- `columns_metadata` (JSONB) - метаданные столбцов (названия, типы данных)
- `total_rows` (int) - общее количество строк
- `additional_params` (JSONB, optional) - дополнительные параметры

Данные отчета

Таблица `table_report_rows` (строки отчета):

- `id` (int, PK, autoincrement) - уникальный идентификатор строки
- `report_id` (int, FK) - ссылка на табличный отчет
- `unique_value` (TEXT) - значение в столбце уникальности (для быстрого поиска и сравнения)
- `created_at` (datetime) - дата создания строки

- `updated_at` (datetime) - дата последнего обновления строки
- `is_deleted` (bool, default=False) - флаг удаления строки (soft delete)

Таблица `table_report_values` (значения ячеек, EAV модель):

- `id` (int, PK, autoincrement) - уникальный идентификатор значения
- `row_id` (int, FK) - ссылка на строку отчета
- `column_name` (VARCHAR(255)) - название столбца
- `value` (TEXT, nullable) - значение ячейки (все данные хранятся как текст)
- `created_at` (datetime) - дата создания значения
- `updated_at` (datetime) - дата последнего обновления значения

Примечание:

- Уникальность строк определяется по значению в указанном столбце (параметр `unique_column` в эндпоинте обновления). Значение хранится в поле `unique_value` для быстрого поиска.
- Данные хранятся в нормализованном виде (EAV модель) для упрощения статистики и запросов по столбцам.
- Все значения хранятся как текст (TEXT) для упрощения структуры и обработки данных.

3. Режимы обновления

- **Полная замена** (`replace`): Сравнение строк по значению в указанном столбце уникальности (`unique_column`). Строки, отсутствующие в новом файле, помечаются как удаленные (soft delete). Новые строки добавляются, существующие обновляются.
- **Дозапись** (`append`): Добавление новых строк к существующим данным. Строки считаются новыми, если значение в столбце уникальности отсутствует в существующих данных.

Параметр `unique_column` : Имя столбца, по которому определяется уникальность строк. Указывается в эндпоинте обновления. Должен существовать в метаданных столбцов отчета.

4. Статистика качества данных

При обновлении отчета рассчитывается статистика на основе сравнения по столбцу уникальности (`unique_column`):

- **Новые строки:** Количество добавленных строк (значение в `unique_column` отсутствует в существующих данных)
- **Обновленные строки:** Количество обновленных строк (значение в `unique_column` существует в обоих наборах данных)
- **Удаленные строки:** Количество удаленных строк (при режиме `replace`, значение в `unique_column` отсутствует в новом файле)
- **Пустые значения по столбцам:**
 - Общее количество пустых значений по каждому столбцу
 - Количество новых пустых значений (в добавленных строках)
- **Уникальные значения по столбцам:**
 - Общее количество уникальных значений по каждому столбцу
 - Количество новых уникальных значений (появившихся в добавленных строках)

ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ

1. Фреймворки

- FastAPI для REST API
- FastStream как плагин (опционально)
- Универсальные хендлеры (UseCases), работающие с обоими фреймворками (свой роутер для каждого фреймворка, в нем вызываем единый хендлер, возвращаем результат в роутер, публикуем по правилам роутера)

2. База данных

- PostgreSQL
- ORM: SQLAlchemy (async)
- Таблица `table_reports` в схеме `controller` для метаданных
- Таблица `table_report_rows` в схеме `controller` для строк отчета
- Таблица `table_report_values` в схеме `controller` для значений ячеек (EAV модель)
- Индексы на `report_id`, `is_deleted` в таблице `table_report_rows` для оптимизации запросов
- Индексы на `row_id`, `column_name` в таблице `table_report_values` для быстрого поиска значений
- Индекс на `unique_value` в таблице `table_report_rows` для быстрого сравнения строк
- Составной индекс на (`row_id`, `column_name`) в таблице `table_report_values` для уникальности значений

3. Архитектура

- Clean Architecture с разделением на слои
- SOLID принципы
- Repository pattern для доступа к данным
- Service layer для бизнес-логики

4. Обработка Excel файлов

- Использование pandas для чтения Excel файлов
- Поддержка форматов .xlsx, .xls
- Обработка больших файлов батчами (batch processing)
- Валидация структуры файла (наличие данных, корректность формата)

СТРУКТУРА КОДА (декомпозиция до файлов)

1. Domain Layer - Модели данных

`app/clients/db/table_report_model.py`

Назначение: Создание нового файла с моделями табличных отчетов

Содержимое:

- `TableReport` - SQLAlchemy модель метаданных табличного отчета
- `TableReportRow` - SQLAlchemy модель строки табличного отчета
- `TableReportValue` - SQLAlchemy модель значения ячейки (EAV модель)
- Использует `ControllerBase` (схема `controller`)- импортируется из `app.clients.db.models`
- Определяет структуру таблиц `table_reports`, `table_report_rows` и `table_report_values` в БД
- Импортирует необходимые типы из SQLAlchemy и базовые классы
- Relationship между `TableReportRow` и `TableReportValue` (one-to-many)

2. Repository Layer - Доступ к данным

`app/clients/db/table_report_repository.py`

Назначение: Создание нового файла с репозиторием для табличных отчетов

Содержимое:

- `TableReportRepository` - класс репозитория с методами:
 - `create(report: TableReport)` -> `TableReport` - создание метаданных отчета
 - `get_by_id(report_id: int)` -> `Optional[TableReport]` - получение метаданных по ID
 - `update(report: TableReport)` -> `TableReport` - обновление метаданных
 - `delete(report_id: int)` -> `None` - удаление отчета (soft delete строк)
 - `create_rows(report_id: int, rows: List[Dict[str, Any]], unique_column: str)` -> `None` - массовое создание строк с их значениями
 - `get_rows(report_id: int, limit: int, offset: int)` -> `List[TableReportRow]` - получение строк с пагинацией (с загрузкой значений)
 - `get_all_rows(report_id: int)` -> `List[TableReportRow]` - получение всех строк (с загрузкой значений)
 - `get_rows_by_unique_value(report_id: int, unique_values: List[str])` -> `List[TableReportRow]` - получение строк по значениям столбца уникальности
 - `get_row_values(row_id: int)` -> `Dict[str, Any]` - получение всех значений строки в виде словаря
 - `get_column_values(report_id: int, column_name: str)` -> `List[Any]` - получение всех значений столбца
 - `replace_rows(report_id: int, rows: List[Dict[str, Any]], unique_column: str)` -> `Tuple[List[Dict], List[Dict], List[Dict]]` - полная замена строк, возвращает (новые, обновленные, удаленные)
 - `append_rows(report_id: int, rows: List[Dict[str, Any]], unique_column: str)` -> `List[Dict]` - дозапись строк, возвращает только новые строки
 - `update_row_values(row_id: int, values: Dict[str, Any])` -> `None` - обновление значений строки
 - `count_rows(report_id: int)` -> `int` - подсчет количества строк
 - `count_empty_values(report_id: int, column_name: str)` -> `int` - подсчет пустых значений в столбце
 - `count_unique_values(report_id: int, column_name: str)` -> `int` - подсчет уникальных значений в столбце
 - `list_reports(user_id: Optional[str], limit: int, offset: int)` -> `List[TableReport]` - список с фильтрацией
- Импортирует `TableReport`, `TableReportRow` и `TableReportValue` из `app.clients.db.table_report_model`
- Импортирует `AsyncSession` из SQLAlchemy

3. Service Layer - Бизнес-логика

`app/services/table_report_crud_service.py`

Назначение: Создание нового файла сервиса для работы с табличными отчетами

Содержимое:

- `TableReportService` - основной сервис для работы с табличными отчетами:
 - `create_report_from_excel(file_path: str, name: str, user_id: str, ...)` -> `TableReport` - создание отчета из Excel
 - `get_report(report_id: int)` -> `TableReport` - получение метаданных отчета
 - `get_report_as_excel(report_id: int)` -> `bytes` - получение отчета в формате Excel
 - `get_report_as_json(report_id: int, limit: int, offset: int)` -> `Dict` - получение отчета в формате JSON
 - `update_report_from_excel(report_id: int, file_path: str, update_mode: str, unique_column: str)` -> `Dict` - обновление отчета из Excel с указанием столбца уникальности
 - `delete_report(report_id: int)` -> `None` - удаление отчета
 - `calculate_quality_stats(report_id: int, new_rows: List[Dict], updated_rows: List[Dict], deleted_rows: List[Dict], unique_column: str)` -> `Dict` - расчет статистики качества
 - `_parse_excel_file(file_path: str)` -> `Tuple[List[Dict], Dict]` - приватный метод парсинга Excel
 - `_validate_excel_file(file_path: str)` -> `None` - приватный метод валидации Excel файла
 - `_validate_unique_column(unique_column: str, columns_metadata: Dict)` -> `None` - приватный метод валидации столбца уникальности
 - `_extract_columns_metadata(df: pd.DataFrame)` -> `Dict` - приватный метод извлечения метаданных столбцов
 - `_convert_value_to_text(value: Any)` -> `str` - приватный метод преобразования значения в текст
 - `_compare_rows_by_unique_column(old_rows: List[TableReportRow], new_rows: List[Dict], unique_column: str)` -> `Tuple[List[Dict], List[Dict], List[Dict]]` - приватный метод сравнения строк (новые, обновленные, удаленные)
 - `_convert_row_to_dict(row: TableReportRow)` -> `Dict[str, Any]` - приватный метод преобразования строки в словарь

4. Use Cases Layer - Сценарии использования

`app/use_cases/table_reports/create_table_report.py`

Назначение: Use case для создания табличного отчета

Содержимое:

- `CreateTableReportUseCase` - сценарий создания табличного отчета:
 - Координация между сервисом и репозиторием
 - Обработка ошибок парсинга Excel
 - Логирование операций

`app/use_cases/table_reports/get_table_report.py`

Назначение: Use case для получения табличного отчета

Содержимое:

- `GetTableReportUseCase` - сценарий получения табличного отчета:
 - Проверка существования отчета
 - Поддержка различных форматов вывода (Excel, JSON)
 - Обработка ошибок доступа
 - Логирование операций

`app/use_cases/table_reports/update_table_report.py`

Назначение: Use case для обновления табличного отчета

Содержимое:

- `UpdateTableReportUseCase` - сценарий обновления табличного отчета:
 - Валидация режима обновления (replace/append)
 - Валидация столбца уникальности (существование в метаданных)
 - Сравнение строк по столбцу уникальности
 - Расчет статистики качества
 - Обработка частичных обновлений
 - Логирование операций

`app/use_cases/table_reports/delete_table_report.py`

Назначение: Use case для удаления табличного отчета

Содержимое:

- `DeleteTableReportUseCase` - сценарий удаления табличного отчета:
 - Проверка прав доступа
 - Soft delete строк
 - Обработка зависимостей
 - Логирование операций

`app/use_cases/table_reports/get_quality_stats.py`

Назначение: Use case для получения статистики качества

Содержимое:

- `GetQualityStatsUseCase` - сценарий получения статистики качества:
 - Расчет статистики по столбцам
 - Подсчет новых/удаленных строк
 - Подсчет пустых и уникальных значений
 - Логирование операций

5. API Layer - Представление

`app/api/schemas.py`

Назначение: Добавление Pydantic схем для табличных отчетов в существующий файл схем

Содержимое:

- `TableReportCreateRequest` - схема запроса на создание отчета (с файлом)
- `TableReportUpdateRequest` - схема запроса на обновление отчета (с файлом, режимом и столбцом уникальности)
- `TableReportResponse` - схема ответа с метаданными отчета
- `TableReportRowResponse` - схема ответа со строкой отчета
- `TableReportDataResponse` - схема ответа с данными отчета (JSON формат)
- `TableReportQualityStatsResponse` - схема ответа со статистикой качества
- `TableReportListResponse` - схема ответа со списком отчетов
- `TableReportListQuery` - схема параметров запроса списка (фильтры, пагинация)

Примечание: Файл уже существует, добавляем новые схемы без изменения существующего кода.

`app/api/handlers/table_report_handlers.py`

Назначение: Создание нового файла с универсальными хендлерами для табличных отчетов

Содержимое:

- `create_table_report_handler(...)` - универсальный хендлер создания отчета
- `get_table_report_handler(report_id: int, format: str, ...)` - универсальный хендлер получения отчета
- `update_table_report_handler(report_id: int, update_mode: str, unique_column: str, ...)` - универсальный хендлер обновления отчета
- `delete_table_report_handler(report_id: int, ...)` - универсальный хендлер удаления отчета
- `get_quality_stats_handler(report_id: int, ...)` - универсальный хендлер получения статистики
- `list_table_reports_handler(...)` - универсальный хендлер списка отчетов

Примечание: Папка `handlers/` не существует, создаем новую структуру. Хендлеры не зависят от конкретного фреймворка и возвращают стандартные структуры данных.

`app/api/routers.py`

Назначение: Добавление FastAPI роутеров для табличных отчетов в существующий файл роутеров

Содержимое:

- `get_table_report_service()` - dependency функция для получения `TableReportService`
- `@router.post("/table-reports")` - создание отчета из Excel файла
- `@router.get("/table-reports/{report_id}")` - получение метаданных отчета
- `@router.get("/table-reports/{report_id}/data")` - получение данных отчета (Excel/JSON)
- `@router.put("/table-reports/{report_id}")` - обновление отчета из Excel файла (параметры: `update_mode`, `unique_column`)
- `@router.delete("/table-reports/{report_id}")` - удаление отчета
- `@router.get("/table-reports/{report_id}/quality-stats")` - получение статистики качества
- `@router.get("/table-reports")` - список отчетов с фильтрацией

Примечание: Файл уже существует, добавляем новые роутеры без изменения существующего кода. Роутеры вызывают универсальные хендлеры из `handlers/table_report_handlers.py`.

6. Миграции БД

`alembic/versions/011_create_table_reports.py`

Назначение: Миграция Alembic для создания таблиц табличных отчетов

Содержимое:

- `upgrade()` - создание таблиц `table_reports`, `table_report_rows` и `table_report_values` в схеме `controller`
- `downgrade()` - удаление таблиц
- Определение всех полей, индексов и ограничений
- Foreign key constraint между `table_report_values.row_id` и `table_report_rows.id`
- Unique constraint на (`row_id`, `column_name`) в таблице `table_report_values`

Примечание: Файл создается новый согласно нумерации миграций (после `010_create_reports_table.py`). Используется EAV модель для хранения значений ячеек.

МАТРИЦА ТЕСТОВ

Unit тесты (tests/unit/table_reports/)

- Создание отчета из Excel файла
- Парсинг Excel файла с различными форматами
- Валидация Excel файла (формат, структура)
- Валидация столбца уникальности (существование в метаданных)
- Сравнение строк по столбцу уникальности (определение новых/обновленных/удаленных)
- Обновление отчета в режиме `replace` с использованием столбца уникальности
- Обновление отчета в режиме `append` с использованием столбца уникальности
- Расчет статистики качества (новые/обновленные/удаленные строки, пустые значения, уникальные значения)
- Получение отчета в формате Excel
- Получение отчета в формате JSON
- Обработка ошибок (отчет не найден, невалидный файл, несуществующий столбец уникальности)

Integration тесты (tests/integration/table_reports/)

- CRUD операции с реальной БД
- Загрузка Excel файла с произвольными столбцами
- Полная замена данных (режим `replace`) с использованием столбца уникальности
- Дозапись данных (режим `append`) с использованием столбца уникальности
- Определение новых/обновленных/удаленных строк по столбцу уникальности
- Проверка целостности данных при обновлении
- Работа с метаданными столбцов (JSONB в `table_reports`)
- Работа с EAV моделью (`table_report_values`) для хранения значений ячеек
- Поиск строк по значениям столбца уникальности
- Пагинация данных отчета
- Расчет статистики качества с реальными данными (включая обновленные строки)
- Экспорт в Excel формат
- Экспорт в JSON формат

API тесты e2e с подключением к БД (tests/integration/table_reports/)

- POST /table-reports - создание из Excel файла
- GET /table-reports/{id} - получение метаданных
- GET /table-reports/{id}/data - получение данных (Excel/JSON)
- PUT /table-reports/{id} - обновление с режимом replace и столбцом уникальности
- PUT /table-reports/{id} - обновление с режимом append и столбцом уникальности
- PUT /table-reports/{id} - валидация столбца уникальности (несуществующий столбец)
- DELETE /table-reports/{id} - удаление отчета
- GET /table-reports/{id}/quality-stats - получение статистики качества (включая обновленные строки)
- GET /table-reports - список с фильтрацией
- Обработка ошибок (404, 422, 500)
- Валидация формата файла

МИГРАЦИЯ БД

Создать Alembic миграцию:

- Таблица `table_reports` в схеме `controller` (метаданные)
- Таблица `table_report_rows` в схеме `controller` (строки отчета)
- Таблица `table_report_values` в схеме `controller` (значения ячеек, EAV модель)
- Foreign key constraints между таблицами
- Индексы для оптимизации запросов и статистики

ДЕКОМПОЗИЦИЯ НА ПОДЗАДАЧИ

REPORT-002-1: Создание моделей `TableReport`, `TableReportRow` и `TableReportValue` в БД (миграция Alembic)

Story Points: 3

Зависимости: -

Описание: Создать миграцию Alembic для таблиц `table_reports`, `table_report_rows` и `table_report_values` (EAV модель) со всеми полями, индексами и ограничениями.

Критерии приемки:

- ☐ Миграция создана в `alembic/versions/`
- ☐ Таблица `table_reports` создана в схеме `controller`
- ☐ Таблица `table_report_rows` создана в схеме `controller` с полем `unique_value`
- ☐ Таблица `table_report_values` создана в схеме `controller` (EAV модель)
- ☐ Все поля определены согласно спецификации (одно поле `value` типа TEXT)
- ☐ Foreign key constraint между `table_report_values.row_id` и `table_report_rows.id`
- ☐ Unique constraint на (`row_id`, `column_name`) в таблице `table_report_values`
- ☐ Индексы созданы для оптимизации запросов и статистики
- ☐ Миграция проходит успешно (`upgrade/downgrade`)

REPORT-002-2: Реализация `TableReportRepository` с базовым CRUD

Story Points: 3

Зависимости: REPORT-002-1

Описание: Реализовать репозиторий для работы с табличными отчетами с базовыми CRUD операциями.

Критерии приемки:

- ☐ `TableReportRepository` реализован в `app/clients/db/table_report_repository.py` с методами: `create()`, `get_by_id()`, `update()`, `delete()`
- ☐ Импортирует `TableReport`, `TableReportRow` и `TableReportValue` из `app.clients.db.table_report_model`
- ☐ Все методы асинхронные с type hints
- ☐ Обработка ошибок с логированием
- ☐ Документация к методам

Тесты (integration тесты в `tests/integration/table_reports/test_table_report_repository.py`):

- ☐ Тест создания отчета через `create()` - отчет сохраняется в БД с правильными данными
- ☐ Тест получения отчета через `get_by_id()` - возвращает правильный отчет
- ☐ Тест получения несуществующего отчета - возвращает None
- ☐ Тест обновления отчета через `update()` - данные обновляются в БД
- ☐ Тест удаления отчета через `delete()` - отчет удаляется из БД (soft delete строк)
- ☐ Тест обработки ошибок БД - исключения логируются и пробрасываются

REPORT-002-3: Реализация методов работы со строками в репозитории

Story Points: 4

Зависимости: REPORT-002-2

Описание: Добавить методы для работы со строками: `create_rows()`, `get_rows()`, `replace_rows()`, `append_rows()`, `count_rows()`.

Критерии приемки:

- ☐ Метод `create_rows()` реализован с поддержкой массовой вставки строк и их значений (EAV модель)

- ☐ Метод `get_rows()` реализован с поддержкой пагинации и загрузкой значений
- ☐ Метод `get_all_rows()` реализован для получения всех строк с загрузкой значений
- ☐ Метод `get_rows_by_unique_value()` реализован для поиска строк по значениям столбца уникальности
- ☐ Метод `get_row_values()` реализован для получения всех значений строки в виде словаря
- ☐ Метод `get_column_values()` реализован для получения всех значений столбца (для статистики)
- ☐ Метод `replace_rows()` реализован с поддержкой столбца уникальности (возвращает новые, обновленные, удаленные)
- ☐ Метод `append_rows()` реализован с поддержкой столбца уникальности (возвращает только новые строки)
- ☐ Метод `update_row_values()` реализован для обновления значений существующей строки
- ☐ Метод `count_rows()` реализован
- ☐ Метод `count_empty_values()` реализован для подсчета пустых значений в столбце
- ☐ Метод `count_unique_values()` реализован для подсчета уникальных значений в столбце
- ☐ Все методы асинхронные с `type hints`
- ☐ Обработка ошибок с логированием
- ☐ Документация к методам

Тесты (integration тесты в `tests/integration/table_reports/test_table_report_rows.py`):

- ☐ Тест создания строк через `create_rows()` - строки и их значения сохраняются в БД (EAV модель)
- ☐ Тест получения строк через `get_rows()` с пагинацией - возвращается правильная часть данных с значениями
- ☐ Тест получения строк по столбцу уникальности - возвращаются правильные строки
- ☐ Тест получения значений строки через `get_row_values()` - возвращается словарь со всеми значениями
- ☐ Тест получения значений столбца через `get_column_values()` - возвращаются все значения столбца
- ☐ Тест полной замены через `replace_rows()` - корректно определяются новые/обновленные/удаленные строки
- ☐ Тест дозаписи через `append_rows()` - новые строки добавляются, существующие не дублируются
- ☐ Тест обновления значений строки через `update_row_values()` - значения обновляются корректно
- ☐ Тест подсчета строк через `count_rows()` - возвращается правильное количество
- ☐ Тест подсчета пустых значений через `count_empty_values()` - возвращается правильное количество
- ☐ Тест подсчета уникальных значений через `count_unique_values()` - возвращается правильное количество
- ☐ Тест сравнения строк по столбцу уникальности - корректно определяются новые/обновленные/удаленные строки

REPORT-002-4: Создание Pydantic схем для API

Story Points: 2

Зависимости: REPORT-002-1

Описание: Создать Pydantic схемы для запросов и ответов API табличных отчетов.

Критерии приемки:

- ☐ `TableReportCreateRequest` создана в `app/api/schemas.py`
- ☐ `TableReportUpdateRequest` создана
- ☐ `TableReportResponse` создана
- ☐ `TableReportRowResponse` создана
- ☐ `TableReportDataResponse` создана
- ☐ `TableReportQualityStatsResponse` создана
- ☐ Все схемы имеют валидацию и документацию
- ☐ `Type hints` для всех полей

Тесты (unit тесты в `tests/unit/table_reports/test_table_report_schemas.py`):

- ☐ Тест валидации `TableReportCreateRequest` с валидными данными - проходит
- ☐ Тест валидации `TableReportCreateRequest` с невалидными данными - выбрасывает `ValidationError`
- ☐ Тест валидации `TableReportUpdateRequest` - проверка режима обновления (`replace/append`) и столбца уникальности
- ☐ Тест валидации `TableReportUpdateRequest` с несуществующим столбцом уникальности - выбрасывает `ValidationError`
- ☐ Тест сериализации `TableReportResponse` из модели - все поля корректно сериализуются
- ☐ Тест валидации `TableReportQualityStatsResponse` - корректная структура статистики (включая обновленные строки)

REPORT-002-5: Реализация парсинга и валидации Excel файлов

Story Points: 3

Зависимости: REPORT-002-2

Описание: Реализовать методы парсинга и валидации Excel файлов в сервисе.

Критерии приемки:

- ☐ Метод `_parse_excel_file()` реализован в `app/services/table_report_crud_service.py`
- ☐ Метод `_validate_excel_file()` реализован
- ☐ Метод `_extract_columns_metadata()` реализован
- ☐ Поддержка форматов `.xlsx`, `.xls`
- ☐ Обработка ошибок парсинга с логированием
- ☐ Документация к методам

Тесты (unit тесты в `tests/unit/table_reports/test_excel_parsing.py`):

- ☐ Тест парсинга валидного Excel файла - данные корректно извлекаются и преобразуются в текст

- ☐ Тест парсинга Excel файла с пустыми ячейками - обрабатывается корректно (пустые значения как пустая строка или NULL)
- ☐ Тест преобразования различных типов данных в текст - числа, даты, булевы значения корректно преобразуются
- ☐ Тест валидации невалидного формата файла - выбрасывается исключение
- ☐ Тест извлечения метаданных столбцов - корректно определяются названия столбцов
- ☐ Тест обработки больших файлов - работает без ошибок памяти

REPORT-002-6: Реализация TableReportService с бизнес-логикой

Story Points: 4

Зависимости: REPORT-002-3, REPORT-002-5

Описание: Реализовать сервис с бизнес-логикой для работы с табличными отчетами.

Критерии приемки:

- ☐ TableReportService создан в app/services/table_report_crud_service.py
- ☐ Метод create_report_from_excel() реализован
- ☐ Метод get_report_as_excel() реализован
- ☐ Метод get_report_as_json() реализован
- ☐ Метод update_report_from_excel() реализован с поддержкой режимов и столбца уникальности
- ☐ Метод _validate_unique_column() реализован для проверки существования столбца
- ☐ Метод _convert_value_to_text() реализован для преобразования значений в текст
- ☐ Метод _compare_rows_by_unique_column() реализован для сравнения строк
- ☐ Валидация данных при создании/обновлении
- ☐ Координация работы с репозиторием
- ☐ Обработка ошибок с логированием
- ☐ Документация к методам

Тесты (unit тесты в tests/unit/table_reports/test_table_report_crud_service.py):

- ☐ Тест создания отчета через сервис - вызывает репозиторий с правильными данными
- ☐ Тест валидации Excel файла при создании - невалидные файлы отклоняются
- ☐ Тест получения отчета в формате Excel - файл генерируется корректно
- ☐ Тест получения отчета в формате JSON - данные сериализуются правильно
- ☐ Тест обновления в режиме replace - старые данные заменяются, удаленные помечаются (soft delete)
- ☐ Тест обновления в режиме append - новые данные добавляются
- ☐ Тест валидации столбца уникальности - несуществующий столбец отклоняется
- ☐ Тест преобразования значений в текст - различные типы данных корректно преобразуются
- ☐ Тест сравнения строк по столбцу уникальности - корректно определяются новые/обновленные/удаленные
- ☐ Тест обработки ошибок - исключения логируются и пробрасываются

REPORT-002-7: Реализация расчета статистики качества

Story Points: 3

Зависимости: REPORT-002-6

Описание: Реализовать расчет статистики качества данных при обновлении отчета.

Критерии приемки:

- ☐ Метод calculate_quality_stats() реализован в app/services/table_report_crud_service.py
- ☐ Подсчет новых строк (по столбцу уникальности)
- ☐ Подсчет обновленных строк (по столбцу уникальности)
- ☐ Подсчет удаленных строк (для режима replace, по столбцу уникальности)
- ☐ Подсчет пустых значений по столбцам (общее и новые)
- ☐ Подсчет уникальных значений по столбцам (общее и новые)
- ☐ Обработка ошибок с логированием
- ☐ Документация к методу

Тесты (unit тесты в tests/unit/table_reports/test_quality_stats.py):

- ☐ Тест подсчета новых строк - возвращается правильное количество (по столбцу уникальности)
- ☐ Тест подсчета обновленных строк - возвращается правильное количество
- ☐ Тест подсчета удаленных строк - возвращается правильное количество (для режима replace)
- ☐ Тест подсчета пустых значений - статистика по столбцам корректна
- ☐ Тест подсчета уникальных значений - статистика по столбцам корректна
- ☐ Тест статистики для новых пустых значений - корректно выделяются новые
- ☐ Тест статистики для новых уникальных значений - корректно выделяются новые

REPORT-002-8: Создание Use Cases для операций с табличными отчетами

Story Points: 3

Зависимости: REPORT-002-6, REPORT-002-7

Описание: Создать Use Cases для каждой операции с табличными отчетами.

Критерии приемки:

- ☐ CreateTableReportUseCase создан в `app/use_cases/table_reports/`
- ☐ GetTableReportUseCase создан
- ☐ UpdateTableReportUseCase создан
- ☐ DeleteTableReportUseCase создан
- ☐ GetQualityStatsUseCase создан
- ☐ Все Use Cases имеют документацию

Тесты (integration тесты в `tests/integration/table_reports/test_table_report_use_cases.py`):

- ☐ Тест `CreateTableReportUseCase` - создает отчет через сервис
- ☐ Тест `GetTableReportUseCase` - получает отчет, обрабатывает ошибки доступа
- ☐ Тест `UpdateTableReportUseCase` - обновляет отчет с валидацией режима
- ☐ Тест `DeleteTableReportUseCase` - удаляет отчет с проверкой прав
- ☐ Тест `GetQualityStatsUseCase` - получает статистику качества
- ☐ Тест обработки ошибок в Use Cases - исключения логируются

REPORT-002-9: Добавление FastAPI роутеров

Story Points: 2

Зависимости: REPORT-002-8

Описание: Добавить FastAPI роутеры для всех операций с табличными отчетами.

Критерии приемки:

- ☐ Роутеры добавлены в `app/api/routers.py`
- ☐ POST `/table-reports` - создание из Excel файла
- ☐ GET `/table-reports/{id}` - получение метаданных
- ☐ GET `/table-reports/{id}/data` - получение данных (Excel/JSON)
- ☐ PUT `/table-reports/{id}` - обновление с режимом
- ☐ DELETE `/table-reports/{id}` - удаление
- ☐ GET `/table-reports/{id}/quality-stats` - получение статистики
- ☐ GET `/table-reports` - список с фильтрацией
- ☐ Обработка ошибок (404, 422, 500)
- ☐ Документация к эндпоинтам

Тесты (e2e тесты в `tests/integration/table_reports/test_table_report_api.py`):

- ☐ Тест POST `/table-reports` - создает отчет, возвращает 201 с данными
- ☐ Тест POST `/table-reports` с невалидным файлом - возвращает 422
- ☐ Тест GET `/table-reports/{id}` - возвращает метаданные, статус 200
- ☐ Тест GET `/table-reports/{id}` для несуществующего - возвращает 404
- ☐ Тест GET `/table-reports/{id}/data?format=excel` - возвращает Excel файл
- ☐ Тест GET `/table-reports/{id}/data?format=json` - возвращает JSON данные
- ☐ Тест PUT `/table-reports/{id}` с режимом `replace` и столбцом уникальности - обновляет отчет, возвращает 200
- ☐ Тест PUT `/table-reports/{id}` с режимом `append` и столбцом уникальности - добавляет данные, возвращает 200
- ☐ Тест PUT `/table-reports/{id}` с несуществующим столбцом уникальности - возвращает 422
- ☐ Тест PUT `/table-reports/{id}` - проверка статистики (новые/обновленные/удаленные строки)
- ☐ Тест DELETE `/table-reports/{id}` - удаляет отчет, возвращает 204
- ☐ Тест GET `/table-reports/{id}/quality-stats` - возвращает статистику качества (включая обновленные строки)
- ☐ Тест GET `/table-reports` с фильтрацией - возвращает отфильтрованный список
- ☐ Тест обработки ошибок - все ошибки возвращают правильные HTTP статусы

ОПРЕДЕЛЕНИЕ ГОТОВНОСТИ (DoD)

- ☐ Все подзадачи выполнены согласно критериям приемки
- ☐ Код отформатирован (black, isort)
- ☐ Нет ошибок линтера (flake8)
- ☐ Проверка типов пройдена (myru)
- ☐ Все тесты проходят (unit + integration + e2e)
- ☐ Покрываемость тестами соответствует требованиям
- ☐ Code review пройден
- ☐ Миграции применены и протестированы
- ☐ Код доступен к запуску, есть файл с зависимостями и докер композ для подключения к бд

ОЦЕНКА ВРЕМЕНИ

Разработка: 19 часов

Тестирование: 2 часа

Общее время: 21 час (Story Points: 21)